

Arquitectura de Software

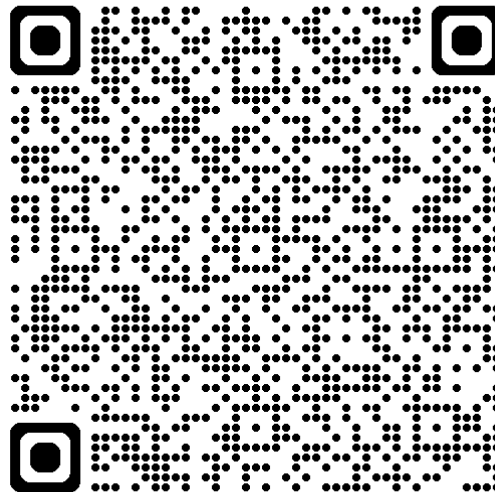
Introducción y reglas del curso

Jaime A. Pavlich Mariscal

Jaime A. Pavlich Mariscal

- Teams y correo
 - jpavlich@javeriana.edu.co

Para agendar cita



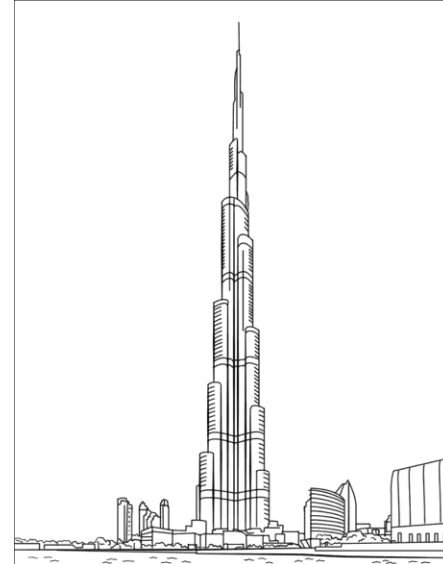
¿Qué es la Arquitectura de Software?



Arquitectura

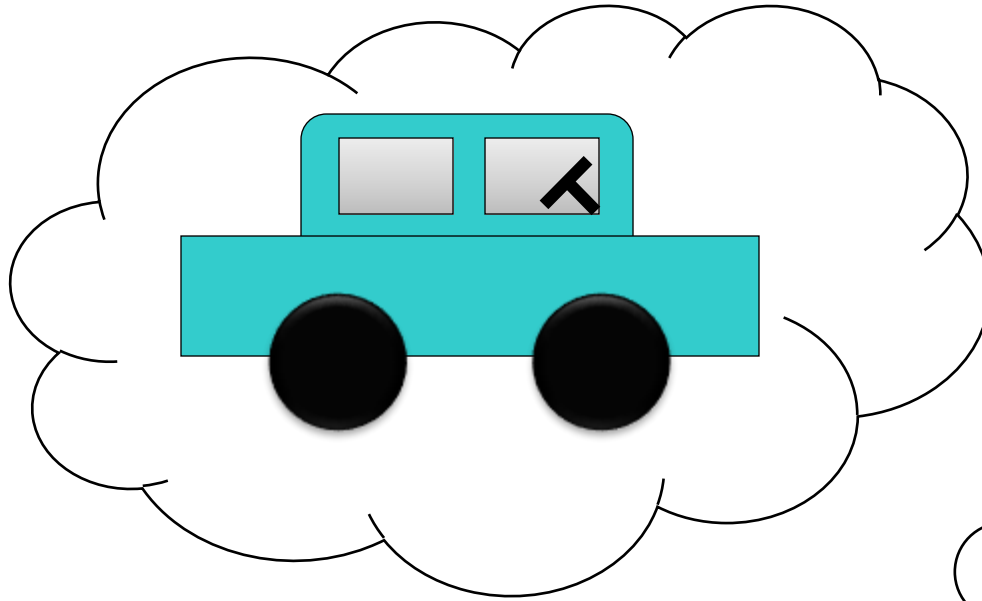


Abstracción de alto nivel de un sistema de software



¿Qué es Abstracción?





- **Esconder detalles irrelevantes**
- **Capturar lo esencial**

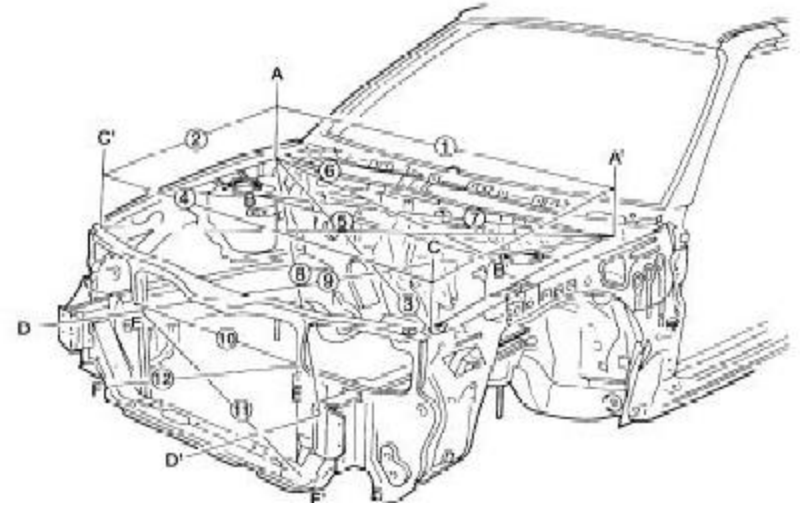


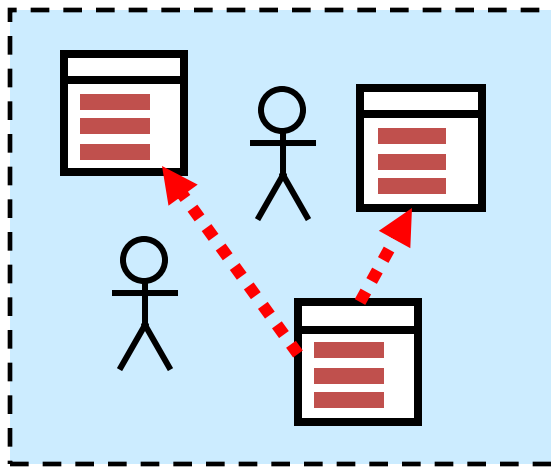
¿Qué es un **Modelo**?



Modelo

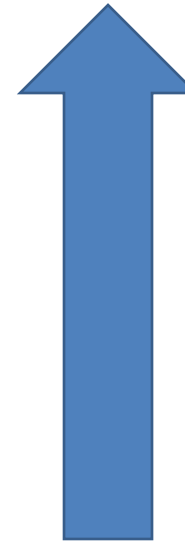
- **Abstracto**
Especificación de una entidad del mundo real
 - Representa propiedades **relevantes** de la entidad
 - Puede ser usado **en lugar** de la entidad para un **propósito** específico





Modelos

Abstracción



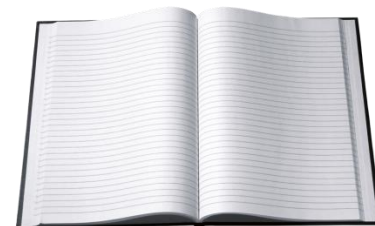
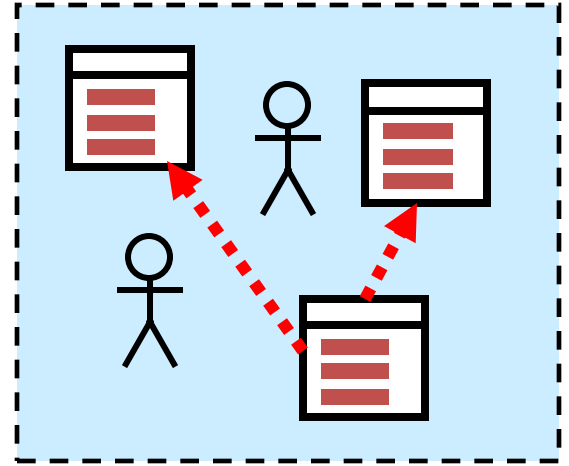
```
public class CourseSection {  
  public class CourseSection {  
    public class CourseSection {  
      public class CourseSection {  
        public CourseDescription getDescription() {  
          <retrieve course description>  
        }  
        public Integer getTerm() {  
          <retrieve term>  
        }  
        ... }  
      }  
    }  
  }  
}
```

Sistema de software

Modelos

Abstracción

Mundo Real



Mundo Real

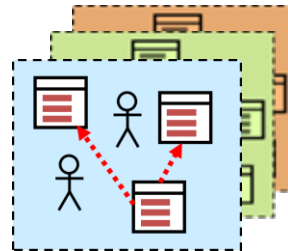


Modelos

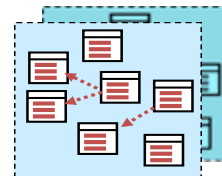
Especificación de
requisitos



"Arquitectura"
(Diseño arquitectónico)



"Diseño"
(Diseño detallado)



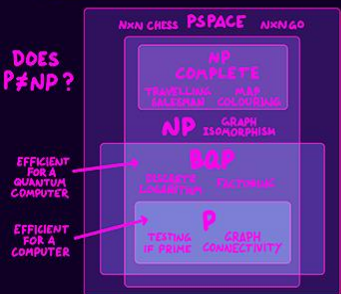
Sistema de software

Programación

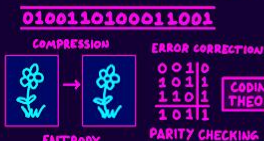


MAP OF COMPUTER SCIENCE

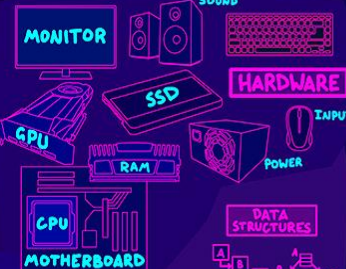
COMPUTATIONAL COMPLEXITY



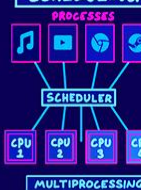
INFORMATION THEORY



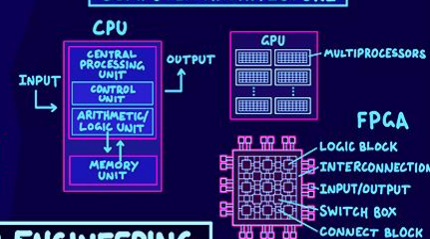
CRYPTOGRAPHY



SCHEDULING



COMPUTER ARCHITECTURE

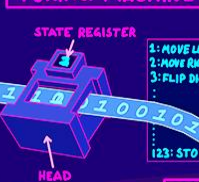


THEORETICAL COMPUTER SCIENCE

COMPUTABILITY THEORY



TURING MACHINE



ALGORITHMS

BUBBLESORT(A)

- GO FROM LEFT TO RIGHT.
- COMPARE EACH PAIR
- IF LEFT ONE HIGHER, SWITCH.
- DO UNTIL NO MORE SWITCHES.

BUBBLE SORT $O(n^2)$

1 2 3 4 5 6 7 8

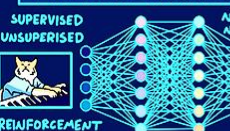
MERGE SORT $O(n \log n)$

1 2 3 4 5 6 7 8

ANALYSIS OF ALGORITHMS

ALGORITHMIC COMPLEXITY

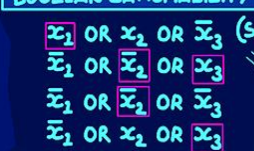
MACHINE LEARNING



ARTIFICIAL INTELLIGENCE



BOOLEAN SATISFIABILITY



SUPER COMPUTING



SOFTWARE ENGINEERING



OPERATING SYSTEMS



NETWORKING



PERFORMANCE



COMPUTER VISION



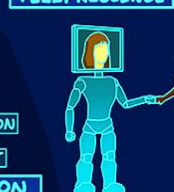
IMAGE PROCESSING



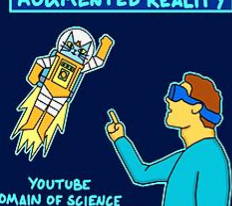
NATURAL LANGUAGE PROCESSING



TELEPRESCENCE



AUGMENTED REALITY



HUMAN COMPUTER INTERACTION



SIMULATION



BIG DATA



INTERNET OF THINGS



BY DOMINIC WALLIMAN ©2017

¿Qué es Arquitectura de software?

- Abstracción de alto nivel
 - Mostrar detalles más "gruesos" del sistema
- Más general que el "diseño"
- Estructura + Comportamiento

Necesaria, pero no suficiente

Integración de la mayoría de lo aprendido en la carrera

Reglas del Curso

Importante

- Indique **oportunamente** al profesor cualquier situación que pueda afectar su desempeño y participación en clases.
 - Situaciones médicas relevantes
 - Trabajo
 - Etc.

¡Hágalo antes de cualquier evaluación o entrega!

Forma de trabajo

- Clases
 - Teoría
 - Práctica
- 6 horas de trabajo independiente
- Diseño Arquitectónico + Código
- Trabajo en Grupo
- Cualquier duda extra
 - Al final de la clase
 - Plataforma virtual
 - Pida cita con el profesor

Evaluaciones

- Parciales (40%)
 - Primer Parcial (20%)
 - Segundo Parcial (20%)
- Tareas, Quices, y Laboratorios (20%)
- Proyecto Semestral (40%)
 - Primera entrega: 15%
 - Segunda entrega: 25%

Evaluaciones “Análogas”

- Algunas evaluaciones podrán ser en papel (Parciales, Quices)
- No se permitirá ningún tipo de dispositivo electrónico
 - Ej: celulares, computadores, audífonos, cámaras, smart glasses, etc.
 - Si Ud. Requiere el uso de algún dispositivo por razones de salud, debe traer el respaldo documental correspondiente
 - Excepción: Smart glasses con lentes de prescripción están prohibidos. Asegúrese de traer gafas comunes y corrientes si así lo necesita.
- Solo se permitirá una “Hoja de Ayuda”
 - Tamaño carta o A4
 - Escrita a mano **por el estudiante**, por ambos lados
- Cualquier material adicional estará prohibido
- El incumplimiento de cualquiera de estas reglas resultará en una nota de 0.0 en la evaluación correspondiente

Entregables

- Las entregas atrasadas (proyecto, tareas, laboratorios, parciales, quices, etc.) tendrán descuento en la nota: 1 punto por hora (redondeado hacia arriba)
 - Ejemplo: 1:20h de atraso -> nota máxima = 3
- Solo por razones debidamente justificadas se puede flexibilizar lo anterior
- Razones de fuerza mayor que impidan la entrega
 - Deben comunicarse al profesor **antes** de la entrega
 - (a no ser que, por fuerza mayor, sea imposible)

Quices

- Quices
 - Pueden ser en cualquier momento
 - Se podrá preguntar cualquier tema visto hasta la clase anterior

Herramientas

- Brightspace (BS)
 - Evaluaciones y entregables
- MS Teams
 - Publicación de notas
 - Material de clase
 - Comunicación con el profesor
- Github (o equivalente)
 - Repositorio de código
 - Documentación
- Herramientas de modelado (Ej. Enterprise Architect)

Proyecto Semestral

Características generales

- Sistema de software no trivial
 - Diseño Arquitectónico
 - Implementación parcial
 - Validación
- Grupos de trabajo: 5-7 personas
 - Máximo 4 grupos de trabajo
 - El profesor tiene la potestad de reasignar integrantes arbitrariamente
- **Todas las entregas deben hacerse antes de la clase donde sustente el primer grupo!**

Características generales del sistema

- Requisitos funcionales relativamente amplios
 - Al menos 5 casos de uso por integrante
 - Cada integrante se hace “dueño” de dichos C.U.
 - Al menos 8 pasos por caso de uso
 - Al menos 1 caso de uso por integrante debe
 - Representar algún proceso "complejo" del negocio (tener varios pasos, escenarios alternativos y excepciones)
 - Tener asociado atributos de calidad no triviales
 - Utilizar algún elemento no trivial de infraestructura
 - Ej: Cola de mensajes, balanceador de carga, etc.
 - Todo lo anterior es negociable, siempre y cuando se demuestre que cada integrante tiene a cargo algo complejo

Requerimientos no funcionales no triviales

- Asuma uno de los siguientes escenarios:
 - Una organización grande con muchos usuarios
 - Una startup que desea crear un producto o servicio con muchos potenciales clientes
- Requisitos fuertes de
 - Desempeño
 - Disponibilidad
 - Seguridad
 - Usabilidad
 - Etc.

Entrega 1

- Primera versión del Software Architecture Description (SAD)
 - ISO/IEC/IEEE 42010
 - Contenidos
 - Especificación de casos de uso y requisitos no funcionales
 - Requisitos Arquitectónicamente Significativos (ASR)
 - Diseño arquitectónico
 - Pruebas de concepto: Código, software configurado para resolver los desafíos técnicos más complejos
 - Prototipo funcional que implemente los casos de uso más complejos por cada integrante
 - Debe implementar los atributos de calidad vistos hasta ese momento en clases
 - Listado de casos de uso + atributos de calidad a implementar en la entrega final
 - Sujeta a aprobación del profesor
 - Compromiso de cumplimiento para próxima entrega

Entrega 2

- Prototipo funcional
 - Implementa casos de uso + atributos de calidad comprometidos en entrega anterior
- 2a versión del SAD
 - Mejoras en el diseño arquitectónico
 - Resultados de validación

Prototipo Funcional

- Por Prototipo Funcional se entiende:
 - Código debidamente documentado
 - Utilizar estándares de programación
 - Código fuente disponible en repositorio
- Despliegue automatizado para cada entrega
 - Cada equipo debe crear un script que
 - Descargue el código fuente desde el repositorio en cada máquina
 - Configure todos sus componentes
 - Ejecute pruebas
 - Despliegue la aplicación y toda su infraestructura

Documentación

- SAD
 - Basado en el formato que entregará el profesor

Sustentación

- ~1h de duración
- Contenidos de la presentación (sin un orden particular)
 - Descripción general de lo entregado
 - Requisitos, diseño, implementación, pruebas
 - Demostración del prototipo
 - Cada integrante debe demostrar su parte
- Debe comenzar en el horario indicado por el profesor

Uso de LLM y otros tipos de IA generativa

- El uso de LLM e IA generativa de imágenes está permitido, tanto para código como para elaborar documentos
- Sin embargo, **cada estudiante debe hacerse responsable de lo que genere la IA**
 - Saber qué significa cada término utilizado en los documentos
 - Saber las razones detrás de cada decisión de diseño
 - Saber qué hace cada línea de código

El profesor podrá verificar lo anterior a su discreción.

La nota podría verse afectada si no se cumple con lo indicado

Calidad y Mejoras

- Las revisiones de calidad de lo entregado son importantes
 - Calidad de código (pruebas)
 - Calidad de los documentos y otros entregables
- Revise la calidad de cada elemento que se haya creado, especialmente si se apoyó en IA
- Evite textos y descripciones excesivamente largos
- Asegurar que todo el equipo entienda la totalidad de los entregables

Equipo de trabajo

EVERY GROUP PROJECT



Trabajo en Equipo

- Defina reglas para el trabajo en grupo
 - Debe incluir causales de expulsión
- Si alguien no trabaja o no cumple, haga valer las reglas establecidas
- Si Ud. es expulsado de un grupo de trabajo, su evaluación a partir de la entrega en que es expulsado es 0,0
- No se permiten cambios de grupo o divisiones de grupo
- Busque espacios para felicitar, premiar o llamar la atención a un compañero. Esto es parte esencial del trabajo en equipo

Herramientas

- Para los documentos
 - Es obligatorio incluir y actualizar una página con el historial de cambios del documento
- Para el código
 - El uso de un sistema de control de versiones es obligatorio (GIT)

Backups

- Algunos entregables deberán ser almacenados en sistemas de la Universidad
- No confíe en la disponibilidad de los servicios
- ¡Tenga un respaldo de todos los archivos, software, etc.!
- **No es excusa válida si algún servicio falla y/o se pierde información**

Tareas urgentes para la próxima clase

- Formar el equipo de trabajo
 - Diligenciar el formato de inscripción de equipos de trabajo (1 por equipo)
 - <https://docs.google.com/forms/d/e/1FAIpQLSdt2GbB2UG1TwZYDZMhebrBed96d7Qpcn6gyGDZ2TQhZdC4Pw/viewform>
- Definir dos posibles ideas de proyectos
 - Listado de todos casos de uso
 - **No olvide que son muchos C.U.!**
 - Justificación de complejidad
 - Requerimientos funcionales
 - Requerimientos no funcionales

¿Preguntas?